

Autonomous BAJA SAEINDIA 2026

MATHWORKS ADVANCED SIMULATION EVENT

Autonomous Navigation Challenge: Guidelines

A Preview of aBAJA 2027



Contents

Contents	2
PART A: OVERVIEW	3
A.1 About the Advanced Simulation Event	3
A.2 Objectives	3
A.3 Event Summary	3
A.4 Simulator Platform	3
A.5 Participation Requirements	4
PART B: SYSTEM CONSTRAINTS AND DESIGN PRINCIPLES	4
B.1 Real-World Reproducibility	4
B.2 Sensor Requirements	4
B.3 Programming Language and Framework	5
B.4 Prohibited Inputs and Techniques	5
PART C: SIMULATION SCENARIO	6
C.1 Environment Description	6
C.2 Scenario Description	7
C.3 Flow of Simulation	7
C.4 Traffic Rules (Strictly Enforced)	8
C.5 Vehicle Model	8
C.6 Environment File Formats	8
PART D: RESTRICTIONS	9
D.1 What Teams CAN Do	9
D.2 What Teams CANNOT Do	9
PART E: SUBMISSION GUIDELINES	10
E.1 Submission Platform	10
E.2 Repository Structure	10
E.3 Complete Deliverables List	10
E.4 Naming Conventions	11
E.5 Submission Deadlines	11
PART F: EVALUATION CRITERIA	11
F.1 Evaluation Overview	11
F.2 Required Log Format	12
F.2.1 Telemetry Log (telemetry_log.csv)	12
F.2.2 Events Log (events_log.json)	13
F.3 Evaluation Metrics and Scoring	14
PART G: ADDITIONAL NOTES AND SUPPORT	15
G.1 Technical Review: Sensor Configuration	15
G.2 Fair Play and Academic Integrity	15
G.3 Support and Resources	15
G.4 Prize Eligibility	16



PART A: OVERVIEW

A.1 About the Advanced Simulation Event

The Advanced Simulation Event is aBAJA SAEINDIA 2026's preview of the aBAJA 2027 competition rules. It gives teams a unique opportunity to get ahead: to work towards next year's autonomous navigation challenge this season, build up their stack early, and understand what the 2027 technical requirements will demand.

Where the Phase 2 Software Virtual World Simulation round validates specific ADAS functions (Automatic Emergency Braking, Lane Keep Assist, and Endurance), this event sets a higher bar. Teams are required to build a complete autonomous driving stack capable of navigating a vehicle from a defined starting point (Point A) to a defined destination (Point B) within a structured urban simulation environment. The vehicle must navigate through intersections, comply with traffic regulations, and avoid static obstacles along the route.

This event is quantifiable and completeness-focused. Performance is measured entirely through structured simulation logs that teams generate and submit. An automated evaluation pipeline processes these logs and computes objective scores based on defined traffic compliance and navigation metrics. There is no presentation component.

NOTE: Participation in the Advanced Simulation Event is optional, and is separate from the Phase 2 Software Virtual World Simulation round.

A.2 Objectives

By the conclusion of this event, teams are expected to have:

- Designed and implemented a complete, end-to-end autonomous driving stack covering: Perception, Localisation, Planning, and Control.
- Imported and configured the simulation environment provided by the technical team into their chosen simulator platform.
- Developed algorithms capable of navigating the ego vehicle autonomously from Point A to Point B without human intervention.
- Demonstrated full compliance with all defined traffic rules throughout the navigation task.
- Generated structured simulation logs in the required format, enabling the automated evaluation pipeline to compute all performance metrics.

A.3 Event Summary

The Advanced Simulation Event comprises a single, integrated autonomous navigation challenge:

S. No	Challenge	Evaluation Mode
1	Autonomous Point-A-to-Point-B Navigation (Urban Environment)	Fully automated: metric-based scoring from submitted simulation logs (see Part F)

A.4 Simulator Platform

The Advanced Simulation Event adopts the same simulator-agnostic approach as the Phase 2 Software Virtual World Simulation round. Teams are free to choose any simulation platform that satisfies the Simulator Requirements document. In addition, the simulator must support:

- Urban road network import via OpenDRIVE (.xodr) or equivalent, including intersection geometry and traffic signal placement.



- Traffic signal simulation with configurable phase timing and signal-state accessible through sensor channels (not ground-truth APIs).
- Static actor placement within the environment.
- Bidirectional data exchange between vehicle model, sensor models, and the team's autonomy software.
- Physics-accurate vehicle dynamics with configurable control interfaces for Throttle, Brake, and Steering.
- The ability to export or stream per-timestep vehicle state data (position, speed, heading, lane deviation) for log generation.

NOTE: Teams must submit their simulator choice and sensor configuration to the Technical Review Team for validation before the evaluation date. Simulators that do not satisfy the requirements document will not be accepted.

NOTE: Only teams that use MathWorks tools (MATLAB, Simulink, RoadRunner, RoadRunner Scenario, etc.) for algorithm creation, simulation, or both will be considered for the prize money for this event. MathWorks tools (MATLAB and Simulink) allow the integration of C++, Python and ROS nodes into Simulink. Apart from Simulating within the MathWorks environment, Algorithms designed in Simulink can be deployed independently into IPG Carmaker, CARLA or other popular simulators, or Co-Simulate with IPG Carmaker and CARLA. The Participating teams can choose to use any of these approaches.

A.5 Participation Requirements

All participating teams are required to:

- Develop a complete, end-to-end autonomous driving stack. Core algorithms and design decisions must be the team's own. Open-source libraries are permitted with proper attribution.
- Import the simulation environment files provided by the Technical Team into their chosen simulator platform (see Part C).
- Submit all required deliverables (videos, structured logs, code) to the team's GitHub repository by the announced submission deadline.
- Undergo a sensor configuration validation review conducted by the Technical Committee.
- Ensure that submitted logs conform exactly to the format defined in Part F. Logs that do not parse correctly will result in a zero score for the affected metrics.

PART B: SYSTEM CONSTRAINTS AND DESIGN PRINCIPLES

B.1 Real-World Reproducibility

Every design decision in the simulation must be implementable on a real physical vehicle. Teams must approach sensor selection, algorithm design, localisation strategies, planning approaches, and vehicle modelling with the same rigour they would apply to an actual autonomous vehicle.

NOTE: Any assumption or shortcut that cannot be physically reproduced on the aBAJA vehicle is not permitted and will be penalised.

B.2 Sensor Requirements

Teams may configure any combination of the following sensor types, provided the configuration is physically plausible for an automotive platform:



Sensor Type	Primary Use	Configurable Parameters
Camera (Monocular / Stereo)	Lane detection, traffic sign/signal recognition, object detection	FOV, update rate, noise, mount position
LiDAR	3D obstacle detection and mapping	Point density, range, noise
RADAR	Object detection: range, velocity	Range, resolution, angular FOV
IMU	Vehicle dynamics: acceleration, angular rate	Mounting position, refresh rate
GNSS (GPS)	Global localisation and positioning	Real-world accuracy modelling required
Additional / Custom Sensors	Any sensor not listed above	Must be submitted for Technical Review and approved prior to use

The following constraints apply to all sensor configurations:

- Sensors must output raw or minimally processed data. Access to perfect ground-truth object states through sensor channels is prohibited.
- Sensor parameterisation must reflect real-world automotive sensor capabilities.
- All sensors must be mounted at physically plausible positions on the vehicle geometry.
- The complete sensor configuration must be documented and submitted for technical review.
- Traffic signal state may only be perceived through camera-based or optical sensor channels, not via ground-truth signal state APIs.

B.3 Programming Language and Framework

Teams are free to implement their stack in any programming language or framework that can interface with the chosen simulator:

- Python (recommended for rapid prototyping, perception pipelines, and data analysis)
- C++ (recommended for performance-critical modules such as real-time planning and control)
- MATLAB / Simulink (suitable for control system design, path planning, and system-level simulation)
- ROS 2 (recommended for modular, node-based system architecture)

B.4 Prohibited Inputs and Techniques

To ensure real-world relevance and engineering integrity, the following are strictly prohibited:

- Using simulator ground-truth object state APIs (positions, velocities, traffic signal state) as direct algorithm inputs.
- Accessing the simulator's internal scene graph or physics engine directly for perception or planning.
- Hard-coding route waypoints, intersection turn sequences, or obstacle positions specific to the evaluation scenario.
- Using pre-recorded sensor data from the same scenario as a replay feed during evaluation.



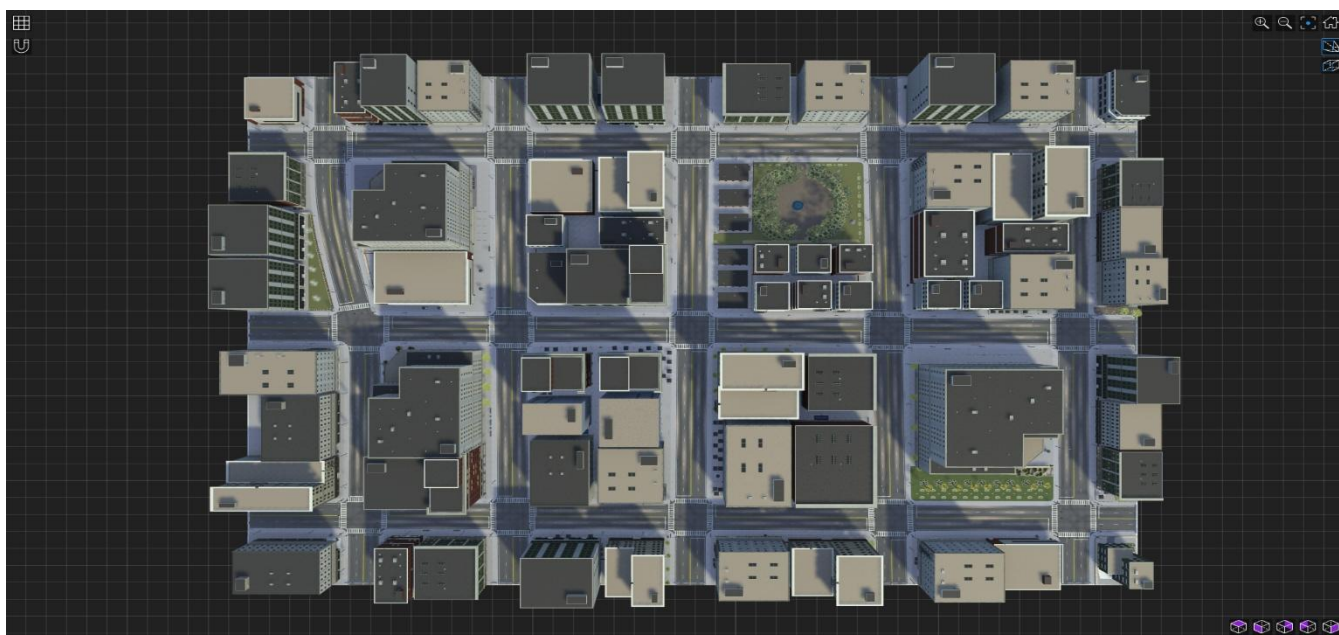
- Any technique that bypasses genuine perception, localisation, planning, or control by exploiting simulator internals.
- Falsifying or post-processing simulation logs before submission. Log integrity will be verified against submitted video evidence.

PART C: SIMULATION SCENARIO

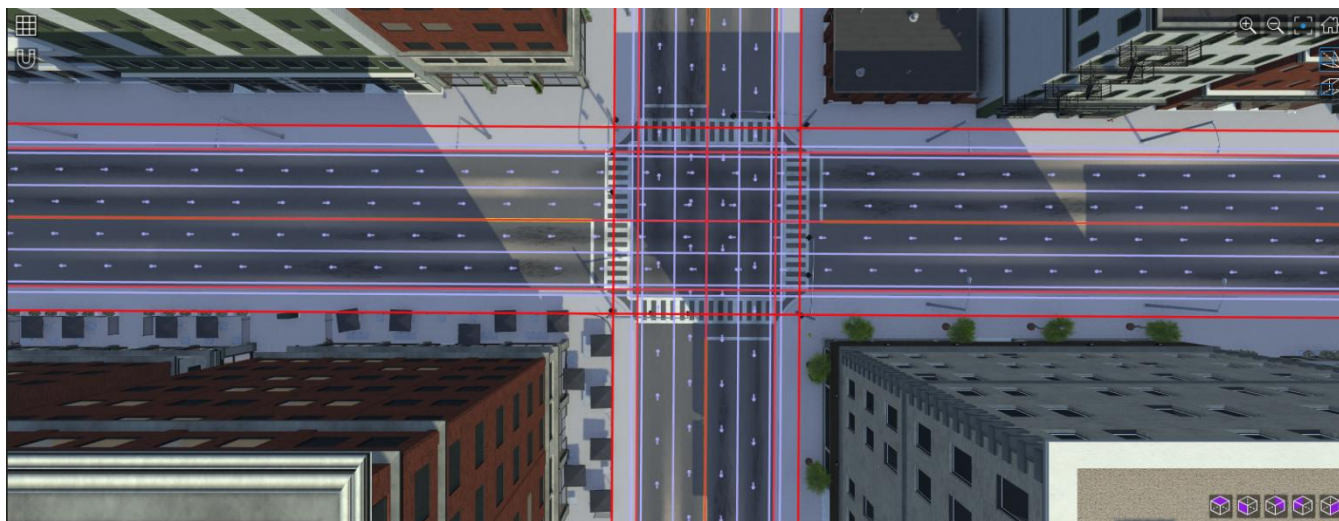
C.1 Environment Description

The mandatory simulation environment is provided by the Technical Team. Teams must import the official environment files into their chosen simulator platform. The environment represents a structured urban city block with the following characteristics:

- 3 parallel horizontal roads and 5 vertical roads, forming a regular grid.



- 4 lanes per road (2 lanes per direction of travel).



- All intersections are 4-way crossings, each equipped with a 4-phase traffic signal system.



- Left-hand drive (LHD) road layout, consistent with Indian traffic convention.
- Clearly marked lane boundaries including solid and dashed lane markings.
- Static obstacles (barriers) placed at defined locations within the environment.

NOTE: Official environment files (OpenDRIVE / FBX / UDatasmith / glTF / RoadRunner Scene) will be released by the Technical Team to all registered teams. A reference video of the environment will also be provided for familiarisation. Teams must not alter the environment geometry or obstacle placement.

C.2 Scenario Description

The navigation task is defined as follows:

- The BAJA vehicle begins from rest, parked on the left side of the road at the designated Point A.
- The vehicle must safely merge from its parked position into the travel lane and navigate autonomously through the road network. The environment contains no dynamic traffic; the navigation task involves static obstacles only.
- The vehicle must traverse multiple intersections, obeying all traffic signals encountered.
- The environment contains static obstacles (barriers). The vehicle must plan and manoeuvre around all obstacles without collision.
- The vehicle must reach the designated Point B and come to a complete stop within the target zone.

NOTE: Exact coordinates for Point A, Point B, are specified in the GitHub repository released to teams.

C.3 Flow of Simulation

- Step 1: Import the official environment files provided by the Technical Team (OpenDRIVE / FBX / UDatasmith / glTF / RoadRunner Scene).
- Step 2: Load the scenario using the Scenario Specification document. Point A, Point B, and obstacle positions are fixed.
- Step 3: Configure the BAJA vehicle dynamics and controllers to match the team's physical vehicle parameters.
- Step 4: Attach and configure sensors. Sensor configuration must have been validated by the Technical Review Team before this step.

- Step 5: Execute the autonomous simulation with log generation active. The vehicle must navigate from Point A to Point B while complying with all traffic rules in Section C.4.
- Step 6: Export the Telemetry Log and Events Log in the formats defined in Part F, Section F.2. Submit all logs with the run videos (POV and Ego Chase) as part of the deliverables.

C.4 Traffic Rules (Strictly Enforced)

Throughout the entire simulation, the vehicle must at all times comply with the following rules. Each rule maps directly to a scored metric in the evaluation pipeline.

ID	Rule	Requirement	Evaluated Metric
TR-1	Speed Limit	The vehicle must not exceed 30 km/h at any point during the run.	Speed Violation Duration (s)
TR-2	Traffic Signal Compliance	The vehicle must stop at red traffic signals, with the front axle behind the stop line.	Red Signal Violation Count
TR-3	Lane Change Discipline	Lane changes must be executed with appropriate turn indicator activation prior to and during the manoeuvre.	Indicator Violation Count
TR-4	Lane Marking Compliance	Solid yellow lane markings must not be crossed under any circumstances.	Yellow Line Violation Count
TR-5	Collision Avoidance	The vehicle must not collide with static obstacles, barriers, or any environmental object.	Collision Count
TR-6	Safe Stopping	The vehicle must come to a complete stop within the designated Point B zone.	Task Completion (Pass/Fail)

C.5 Vehicle Model

Teams must configure a vehicle model that accurately represents the aBAJA vehicle's physical characteristics:

- Vehicle mass, wheelbase, and track width must reflect the team's actual aBAJA build specifications.
- Tyre models must be physically representative. Standard automotive tyre models are acceptable.
- The 3D vehicle geometry/mesh used in the simulation must be submitted as part of the deliverables.

NOTE: Teams may use simulator-provided vehicle templates as a starting point, but must parameterise them to approximate the aBAJA vehicle's actual dynamics. Using generic templates without parameterisation will be penalised.

C.6 Environment File Formats

The Technical Team will release the official environment in the following formats:

- OpenDRIVE (.xodr) for road network definition (primary format, preferred)
- FBX for 3D environment geometry
- Udatasmith for Unreal Engine-based simulators
- glTF / GLB for 3D asset import
- RoadRunner Scene (.rrscene / .rrscenario) for MathWorks RoadRunner-based simulators

NOTE: Teams must use the official environment files without modification. Any alteration to environment geometry, obstacle placement, or signal configuration will be treated as a restriction violation and may result in disqualification.



PART D: RESTRICTIONS

The following restrictions govern all simulation work submitted. Violations may result in penalties, score reductions, or disqualification.

ID	Restriction	Detail
R1	No Ground-Truth Inputs	Teams must not use perfect ground-truth object positions, velocities, traffic signal state, or map data as direct algorithm inputs. All perception must be derived from realistic sensor models.
R2	No Simulator-Only Shortcuts	Any technique that cannot be physically replicated on a real vehicle is prohibited, including simulator APIs that expose privileged world-state data.
R3	Sensor Data Only	Algorithm inputs must come exclusively from configured sensor models. No direct scene-graph or physics-engine access is permitted.
R4	No Route Hard-Coding	Teams must not hard-code route waypoints, intersection turn sequences, or obstacle positions from the evaluation scenario.
R5	No Environment Modification	Teams must not alter the official environment files, obstacle placement, or traffic signal configuration provided by the Technical Team.
R6	Sensor Config Validation	The sensor configuration must be submitted and validated by the Technical Review Team prior to the evaluation run.
R7	No Log Falsification	Simulation logs must be generated directly by the team's simulation pipeline. Post-processing or falsification is prohibited. Log entries will be cross-checked against submitted video.
R8	Reproducibility Required	Submitted code and configuration must produce consistent results when re-run by the Technical Committee.
R9	Version Control Mandatory	All source code, configuration files, and scenario files must be tracked in the team's GitHub repository with meaningful commit history.
R10	Individual Team Work	Cross-team sharing of core algorithm code is not permitted. Open-source libraries and frameworks are allowed with proper attribution.

D.1 What Teams CAN Do

- Use any compliant simulator platform that meets the Simulator Requirements specification.
- Use open-source perception and planning libraries (OpenCV, PCL, YOLO, Nav2, OMPL, etc.) with proper attribution.
- Use HD Maps, OpenDRIVE lane graphs, or custom road representations for planning.
- Tune sensor parameters (FOV, noise, update rate) within physically plausible ranges.

D.2 What Teams CANNOT Do

- Modify the official environment files or alter obstacle placement.



- Share core algorithm code with other registered teams.
- Submit simulation results not produced by the team's own algorithm during the current season.

PART E: SUBMISSION GUIDELINES

E.1 Submission Platform

All deliverables must be submitted via the team's GitHub repository. Each team will receive access credentials from the Technical Committee. The repository must:

- Be kept private until the official submission deadline.
- Maintain a clear and meaningful commit history throughout the development season.
- Contain a README.md at the root level with an overview of the repository structure, system architecture, and run instructions.

E.2 Repository Structure

Teams must organise their repository using the following folder structure:

```
TeamID_TeamName/
├── src/                <- All source code (perception, localisation, planning,
control)
├── config/             <- Sensor configs, vehicle parameters, map files
├── scenarios/          <- Scenario files
├── results/
│   ├── Navigation/    <- POV video + Ego Chase video + BEV trajectory plot
│   └── logs/          <- telemetry_log.csv + events_log.json
├── docs/              <- System architecture doc, sensor config report
└── README.md          <- Repository overview and run instructions
```

E.3 Complete Deliverables List

S.No	Deliverable	Description	Format
1	Source Code Repository	Complete autonomous driving stack (perception, localisation, planning, control)	GitHub (version-controlled)
2	Navigation Video	Ego-vehicle POV video of the full A-to-B run, clearly showing obstacle avoidance, intersection behaviour, and signal compliance	.mp4 (share link if >100 MB)
3	Ego Chase Video	Third-person ego-chase camera view of the full A-to-B run, clearly showing vehicle behaviour, obstacle manoeuvres, and intersection navigation	.mp4 (share link if >100 MB)
4	BEV Trajectory Plot	Bird's Eye View of ego trajectory overlaid on the map, colour-coded by vehicle velocity	.png
5	Telemetry Log	Per-timestep vehicle state log in the exact CSV format defined in Part F, Section F.2	telemetry_log.csv
6	Events Log	Discrete-event log in the exact JSON format defined in Part F, Section F.2	events_log.json

7	System Architecture Document	Block diagram and written description of the full autonomous stack, including module interfaces and data flows	.pdf
8	Sensor Configuration File	Documented sensor placement, FOV, noise parameters, and rationale for each sensor selected	.pdf

NOTE: The Telemetry Log and Events Log are the primary evaluation inputs. Any submission missing these files, or with logs that do not parse against the schema defined in Part F, will receive a zero score for all automated metrics. Teams should validate their logs locally against the provided log validator script before submission.

E.4 Naming Conventions

All files must follow the convention: TeamID_TeamName_[Deliverable]. Examples:

- TeamID_TeamName_NavigationVideo.mp4
- TeamID_TeamName_EgoChaseVideo.mp4
- TeamID_TeamName_BEV_TrajectoryPlot.png
- TeamID_TeamName_telemetry_log.csv
- TeamID_TeamName_events_log.json
- TeamID_TeamName_SystemArchitecture.pdf
- TeamID_TeamName_SensorConfig.pdf

E.5 Submission Deadlines

The Submission deadline is on 24th July 2026 23:59 IST.

Teams are advised to:

- Submit source code and all result files by the announced code submission deadline.
- Ensure the sensor configuration has been validated by the Technical Review Team at least two weeks before the evaluation date.
- Validate logs locally using the provided log validator script before submission.
- Not make changes to the main branch of the repository after the submission deadline.

PART F: EVALUATION CRITERIA

F.1 Evaluation Overview

All scoring in the Advanced Simulation Event is fully automated. Teams submit structured simulation logs alongside their run video, and the Technical Committee's evaluation pipeline parses these logs to compute objective performance scores. There is no presentation component.

This approach ensures every team is measured against the same criteria under identical conditions, with no subjective interpretation. Scores are computed by running the log analysis script against the submitted files and generating a scorecard for each team. Perception-derived log fields (e.g., signal_detected, lateral_deviation_m) are independently verified by the Technical Committee against the official signal phase timing, the submitted videos, and re-runs of the submitted code. Discrepancies between logged values and verified behaviour are treated as log falsification under Part G.2.

NOTE: The evaluation pipeline, log validator script, and a sample log from a reference run will be released to all registered teams at least four weeks before the submission deadline. Teams are expected to use these tools to validate their logs before submitting.

F.2 Required Log Format

Teams must generate two log files during the simulation run: a Telemetry Log (CSV) capturing per-timestep vehicle state, and an Events Log (JSON) capturing discrete driving events. Both files must conform exactly to the schemas defined below.

F.2.1 Telemetry Log (telemetry_log.csv)

The Telemetry Log must be generated at a minimum frequency of 10 Hz (one row every 0.1 s). Each row represents one timestep snapshot of the vehicle's state. Required columns (in this exact order):

Column Name	Data Type	Description
timestamp_s	float	Simulation time in seconds from start of run (e.g., 0.0, 0.1, 0.2...)
ego_x	float	Ego vehicle X position in world coordinates (metres)
ego_y	float	Ego vehicle Y position in world coordinates (metres)
ego_speed_kmh	float	Ego vehicle speed in km/h (always non-negative)
ego_heading_deg	float	Ego vehicle heading in degrees (0-360, where 0 = North and angles increase clockwise: 90 = East)
lateral_deviation_m	float	Lateral distance from the centre of the current lane (metres, signed: positive = right of centre)
indicator_state	string	Turn signal state: none, left, or right
vehicle_length_m	float	Overall length of the ego vehicle including bodywork (metres). Constant for the entire run.
vehicle_width_m	float	Overall width of the ego vehicle including bodywork (metres). Constant for the entire run.
ego_origin_ref	string	Reference point on the vehicle used for reporting ego_x / ego_y: rear_axle_center, geometric_center, or front_axle_center. Must be constant for the entire run and documented in the Sensor Configuration report.
collision_flag	int	1 if a collision contact event is active at this timestep, 0 otherwise
signal_detected	string	Signal state as perceived by the vehicle's sensors: red, green, yellow, or none
task_complete	int	1 if the vehicle is within the Point B zone and speed < 0.5 km/h, 0 otherwise

Example header and first two data rows:

```
timestamp_s,ego_x,ego_y,ego_speed_kmh,ego_heading_deg,lateral_deviation_m,indicator_state
,vehicle_length_m,vehicle_width_m,ego_origin_ref,collision_flag,signal_detected,task_comple
te
0.0,12.40,85.20,0.0,90.0,0.02,none,2.10,1.40,rear_axle_center,0,none,0
0.1,12.40,85.21,1.2,90.1,0.03,none,2.10,1.40,rear_axle_center,0,none,0
```

F.2.2 Events Log (events_log.json)

The Events Log captures discrete driving events during the run. It must be a valid JSON file containing a single top-level array of event objects. Required fields per event:

Field	Data Type	Description
timestamp_s	float	Simulation time in seconds when the event occurred
event_type	string	One of: lane_change_start, lane_change_end, lane_following, right_turn, left_turn, intersection_entry, intersection_exit, collision_start, collision_end
ego_x	float	Ego vehicle X position at event time (metres)
ego_y	float	Ego vehicle Y position at event time (metres)
details	object	Event-specific detail fields (see table below)

Required detail fields by event type:

event_type	details fields required
lane_change_start	{ "from_lane": "<lane_id>", "to_lane": "<lane_id>", "indicator_active": true/false }
lane_change_end	{ "from_lane": "<lane_id>", "to_lane": "<lane_id>", "success": true/false }
lane_following	{ "ego_speed_kmh": <float> }
right_turn	{ "intersection_id": "<id>", "ego_speed_kmh": <float> }
left_turn	{ "intersection_id": "<id>", "ego_speed_kmh": <float> }
intersection_entry	{ "intersection_id": "<id>", "signal_state_detected": "red/green/yellow/none" }
intersection_exit	{ "intersection_id": "<id>" }
collision_start	{ "obstacle_id": "<id>", "ego_speed_kmh": <float> }
collision_end	{ "obstacle_id": "<id>", "duration_s": <float> }

Example events_log.json:

```
[
  { "timestamp_s": 4.2, "event_type": "lane_change_start", "ego_x": 15.3, "ego_y": 92.1,
    "details": { "from_lane": "R1_L2", "to_lane": "R1_L1", "indicator_active": true } },
  { "timestamp_s": 41.0, "event_type": "right_turn", "ego_x": 120.6, "ego_y": 150.2,
    "details": { "intersection_id": "I5", "ego_speed_kmh": 12.4 } }
]
```


NOTE: A log validator script will be released by the Technical Team. Run it before submission: `python3 validate_logs.py telemetry_log.csv events_log.json`. Any schema validation errors must be resolved before submitting.

F.3 Evaluation Metrics and Scoring

The evaluation pipeline computes the following metrics from submitted logs. All metrics are computed programmatically with no manual interpretation. The final score is the sum of all metric scores, subject to penalties.

Metric	Computed From	Scoring	Max Score
Task Completion	telemetry_log.csv: task_complete = 1 (vehicle stopped within the Point B zone)	Pass = 20 pts; Fail = 0 pts. Gates further positive scoring.	20
Navigation Time	telemetry_log.csv: timestamp_s of the first row where task_complete = 1	Score = $15 \times (1 - (T - T_{\min}) / (T_{\max} - T_{\min}))$, clamped to [0, 15]. T_min and T_max will be published together with the evaluation pipeline script release.	15
Collision Penalty	telemetry_log.csv: collision_flag grouped into collision events	-5 pts per collision event (minimum score 0)	0 (penalty)
Speed Limit Compliance	telemetry_log.csv: rows where ego_speed_kmh > 30	10 pts minus 1 pt per second above 30 km/h (minimum 0)	10
Red Signal Compliance	telemetry_log.csv: ego position, heading, and vehicle dimensions evaluated by the pipeline against the official stop line locations and ground-truth signal phase timing	10 pts minus 5 pts per red signal violation (minimum 0)	10
Lane Marking Compliance	telemetry_log.csv: ego position, heading, and vehicle dimensions evaluated against the official lane marking geometry; crossings grouped into events	10 pts minus 3 pts per yellow line crossing event (minimum 0)	10
Indicator Compliance	events_log.json: lane_change_start with indicator_active = false	5 pts minus 2 pts per lane change without indicator (minimum 0)	5
Minimum Obstacle Distance	telemetry_log.csv: minimum clearance computed by the pipeline from ego position, heading, and vehicle dimensions against the official obstacle positions	5 pts if ≥ 1.0 m; 3 pts if ≥ 0.5 m; 1 pt if ≥ 0.2 m; 0 pts if < 0.2 m	5
Lane Centering Accuracy (RMSE)	telemetry_log.csv: RMSE of lateral_deviation_m	5 pts if RMSE ≤ 0.10 m; 3 pts if ≤ 0.20 m; 1 pt if ≤ 0.35 m; 0 pts if > 0.35 m	5
TOTAL (before penalties)			80

A team that does not achieve task completion receives 0 for Navigation Time but may still earn points on compliance metrics computed from the partial run data in the Telemetry Log. The official obstacle placement guarantees that a clearance of at least 1.0 m is achievable along every valid route, so the Minimum Obstacle Distance thresholds never penalise unavoidable proximity.

NOTE: The evaluation pipeline script will be released to teams, enabling local score previewing before final submission. There is no re-evaluation or appeals process for log parsing failures caused by non-conforming log formats.

PART G: ADDITIONAL NOTES AND SUPPORT

G.1 Technical Review: Sensor Configuration

All teams must submit their sensor configuration for review by the aBAJA Technical Committee. The review will assess:

- Physical plausibility of sensor types and mounting positions.
- Whether sensor parameterisation reflects real-world automotive sensor capabilities.
- Absence of prohibited ground-truth data feeds.
- Presence of a traffic signal detection mechanism through sensor channels, not ground-truth APIs.

The Technical Committee will return feedback within the window specified in the competition calendar. Teams must address all flagged issues before the evaluation run.

G.2 Fair Play and Academic Integrity

All work submitted must represent the genuine efforts of the registered team. The Technical Committee reserves the right to:

- Request a live re-run of the submitted simulation to verify log reproducibility.
- Cross-check log data against the submitted video to detect falsified entries.
- Investigate suspected violations of the restrictions outlined in Part E.
- Run the submitted code independently on the evaluation infrastructure to verify consistent results.

Teams found to have violated academic integrity standards, including log falsification, post-processing of simulation data, or cross-team algorithm sharing, will be disqualified from the 2026 competition and may be barred from future aBAJA events.

G.3 Support and Resources

The following support resources are available to all registered teams:

- Simulator Workshops: Technical Committee-led sessions on environment import, sensor configuration, and co-simulation setup. Dates announced on the aBAJA SAEINDIA forum.
- Official Environment Files: Released in OpenDRIVE, FBX, UDatasmith, glTF, and RoadRunner Scene formats to all registered teams.
- Reference Video: A reference video of the mandatory navigation scenario will be released for environment familiarisation.
- Scenario Specification Document: Exact Point A, Point B coordinates, and obstacle placement details released within the announced timeline.
- Log Validator Script: A Python validation script released at least four weeks before the submission deadline. Teams must use this to validate both log files before submitting.
- Evaluation Pipeline Script: The exact scoring script used by the Technical Committee will be released to teams, enabling local score previewing before final submission.



- Technical Q&A Channel: A dedicated support channel on the aBAJA SAEINDIA forum maintained throughout the Phase 2 season.

G.4 Prize Eligibility

Only teams that use MathWorks tools (MATLAB, Simulink, RoadRunner, RoadRunner Scenario, etc.) for algorithm creation, simulation, or both will be considered for the prize money for this event.

MathWorks tools (MATLAB and Simulink) allow the integration of C++, Python and ROS nodes into Simulink. Apart from Simulating within the MathWorks environment, Algorithms designed in Simulink can be deployed independently into IPG Carmaker, CARLA or other popular simulators, or Co-Simulate with IPG Carmaker and CARLA. The Participating teams can choose to use any of these approaches.

OUR SPONSORS

Title (mBAJA & eBAJA)	Title (aBAJA)	Principal Host	Platinum	Diamond Host
 High Performance Patrol				
 ए आर ए आई Progress through Research				
Platinum Host	Gold	Silver		
	     			
Gold Host	Hospitality Partner	Bronze	Event	
			      	
Event	Start Up Associate	Inkind Sponsor		
		    		
	Inkind Sponsor	Assessment Partner		
		   		
Media Partner	Supported By			
   	     			

